

Operating Systems

Project Checkpoint 4

112070002 張博安

Table of Contents

1	Screenshots for Compilation	2
2	Screenshots and Explanation	2
2.1	Producer 1	3
2.2	Producer 2	4
2.3	Consumer	6
2.4	Unfairness	7

1 Screenshots for Compilation

```
boanz@BENSONCH /cygdrive/d/os/ppc4
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
make: *** [Makefile:25: clean] Error 1

boanz@BENSONCH /cygdrive/d/os/ppc4
$ make
sdcc -c test3threads.c
test3threads.c:51: warning 158: overflow in implicit constant conversion
sdcc -c preemptive.c
preemptive.c:216: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
preemptive.c:274: warning 158: overflow in implicit constant conversion
sdcc -o test3threads.hex test3threads.rel preemptive.rel
```

Figure 1: Compilation result of `make clean` and `make`

2 Screenshots and Explanation

	value	Global	Global Defined In Module
	-----	-----	-----
C:	00000081	_Producer1	test3threads
C:	000000CF	_Producer2	test3threads
C:	0000011D	_Consumer	test3threads
C:	00000167	_main	test3threads
C:	00000182	__sdcc_gsinit_startup	test3threads
C:	00000186	__mcs51_genRAMCLEAR	test3threads
C:	00000187	__mcs51_genXINIT	test3threads
C:	00000188	__mcs51_genXRAMCLEAR	test3threads
C:	00000189	_timer0_ISR	test3threads
C:	0000018F	_Bootstrap	preemptive
C:	000001B8	_myTimer0Handler	preemptive
C:	00000234	_ThreadCreate	preemptive
C:	000002A7	_ThreadYield	preemptive
C:	000002FB	_ThreadExit	preemptive

Figure 2: Function address list

2.1 Producer 1

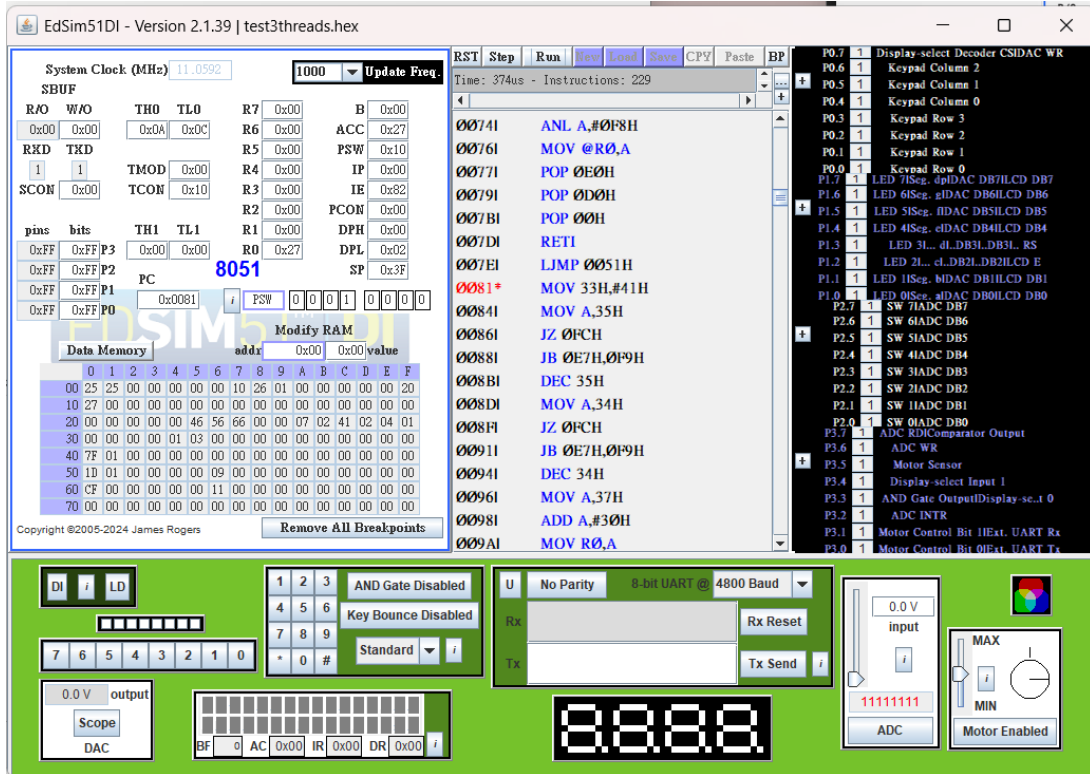


Figure 3: Screenshot for first `Producer1` call

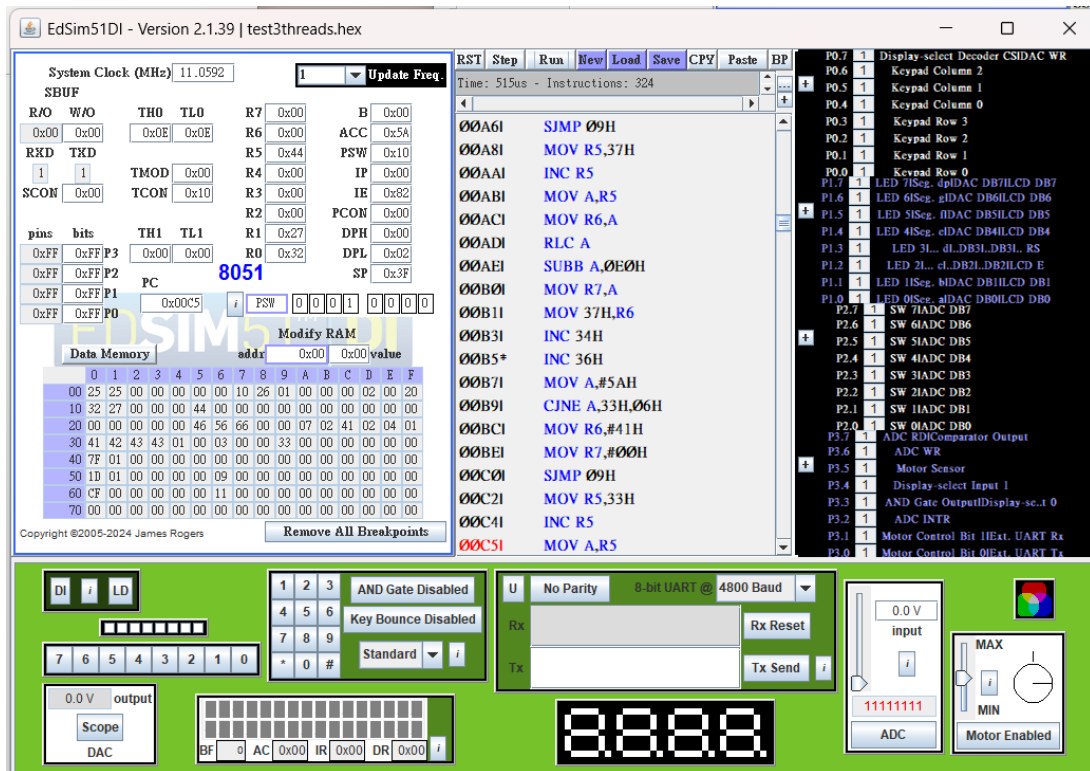


Figure 4: Screenshot before the `Producer1` was preempted

The `Producer1` function's code address starts at `0x0081`. The semaphore values of `mutex`, `empty`, and `full` are stored at `0x34`, `0x35`, and `0x36`, respectively. For the first run, since the value of `empty` is 3, `Producer1` enters the critical section and pushes three characters (A, B, C) into the buffer (ranging from `0x30` to `0x32`). After that, the values of `mutex`, `empty`, and `full` become 1, 0, and 3, respectively.

2.2 Producer 2

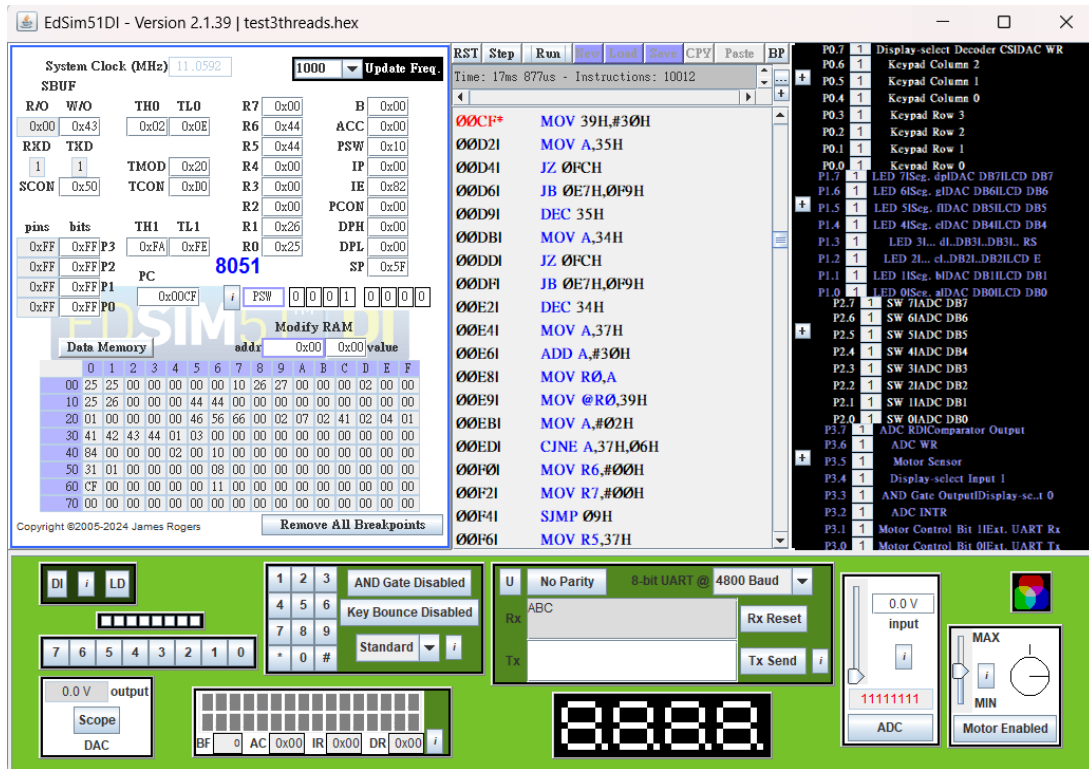


Figure 5: Screenshot for first `Producer2` call

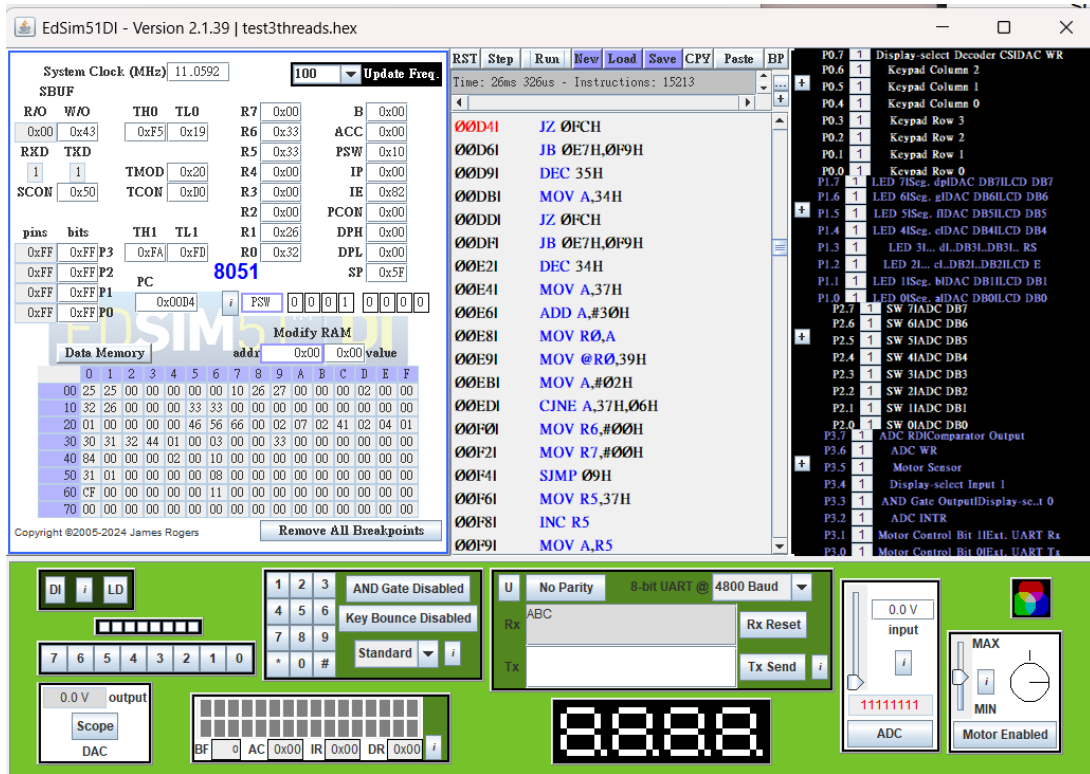


Figure 6: Screenshot before the `Producer2` was preempted

The `Producer2` function's code address starts at `0x00CF`. Similar to `Producer1`, since the value of `empty` is 3 when `Producer2` first runs, it enters the critical section and pushes three digits (0, 1, 2) into the buffer. After that, the values of `mutex`, `empty`, and `full` become 1, 0, and 3, respectively.

2.3 Consumer

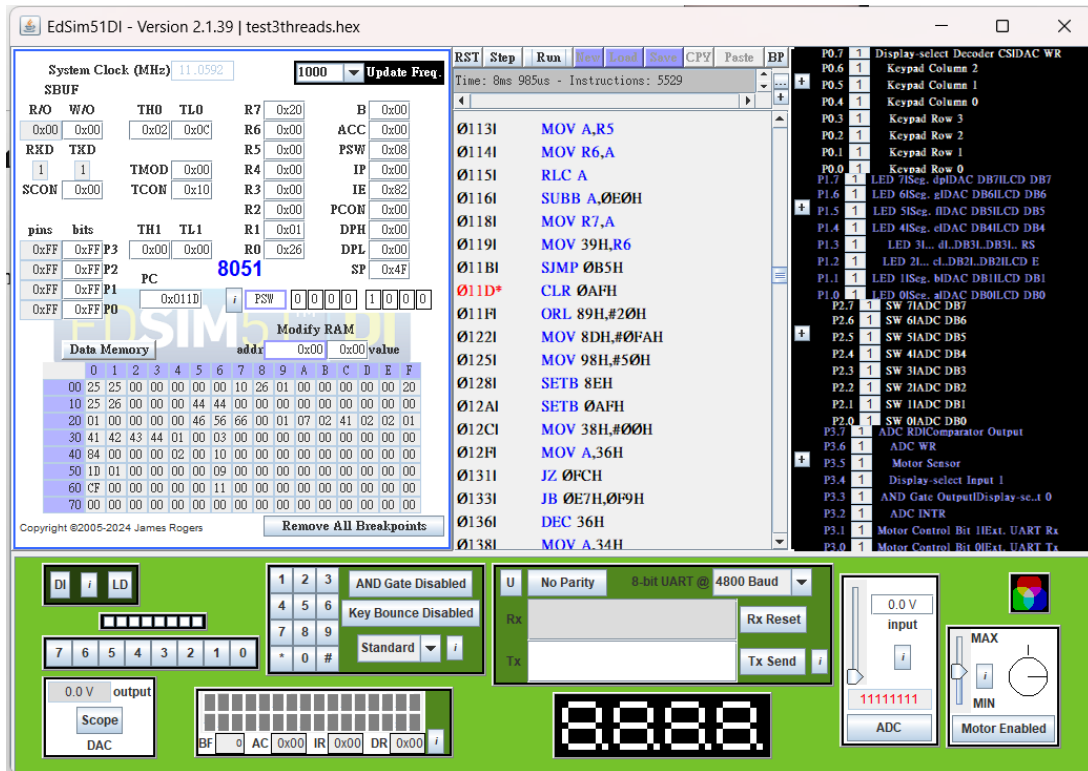


Figure 7: Screenshot for first [Consumer](#) call

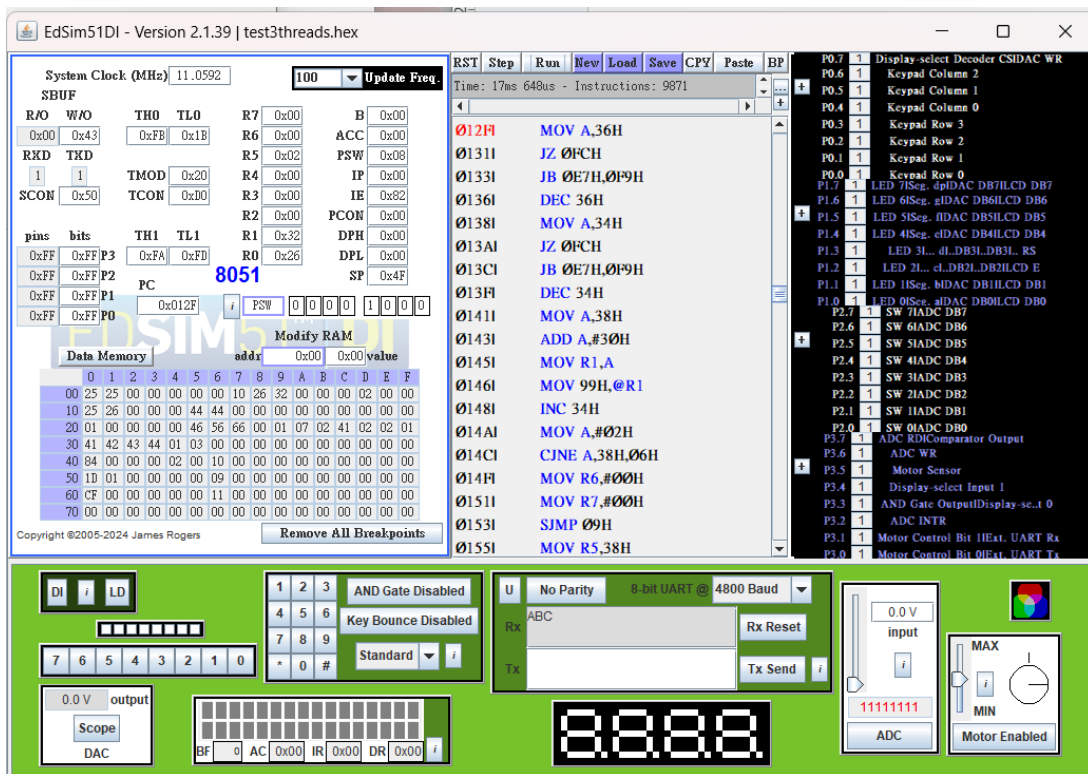


Figure 8: Screenshot before the [Consumer](#) was preempted

The **Consumer** function's code address starts at 0x011D. When **Consumer** runs, the buffer is full. It enters the critical section and sends each byte from the buffer to SBUF for UART transmission, then waits for the TI flag to confirm completion before moving to the next item. After all items are consumed, the values of **mutex**, **empty**, and **full** become 1, 3, and 0, respectively. The **Consumer** then waits to be preempted by the timer.

2.4 Unfairness

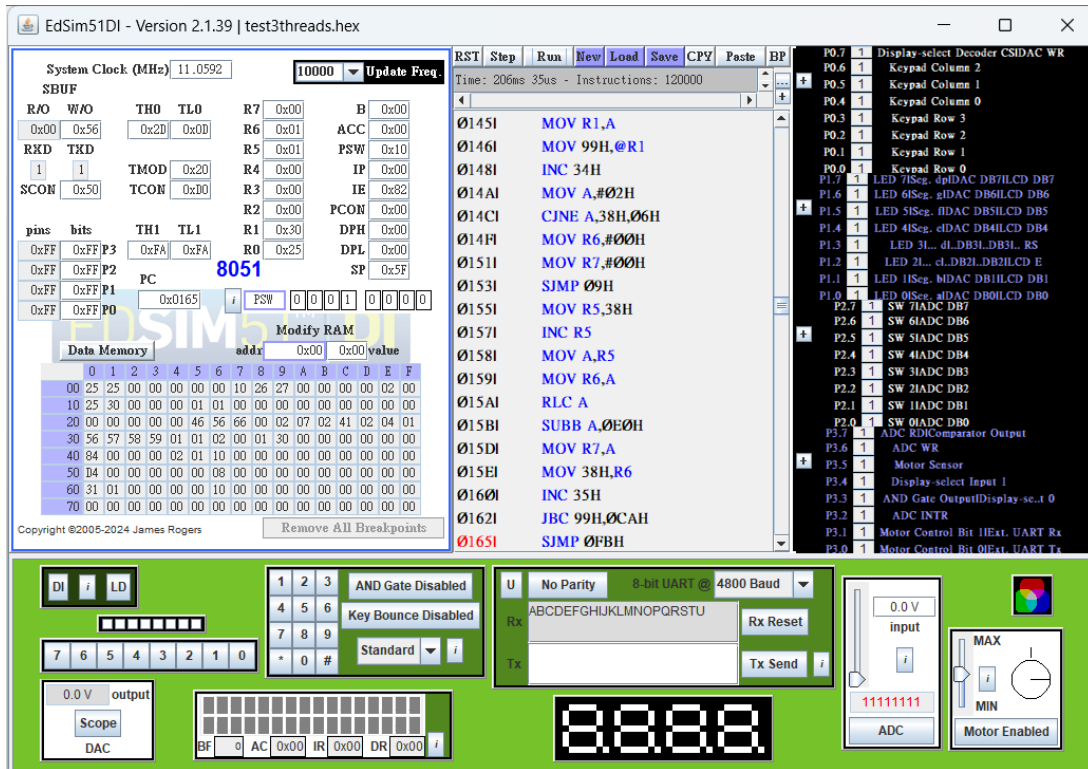


Figure 9: Unfair condition for **Producer2**

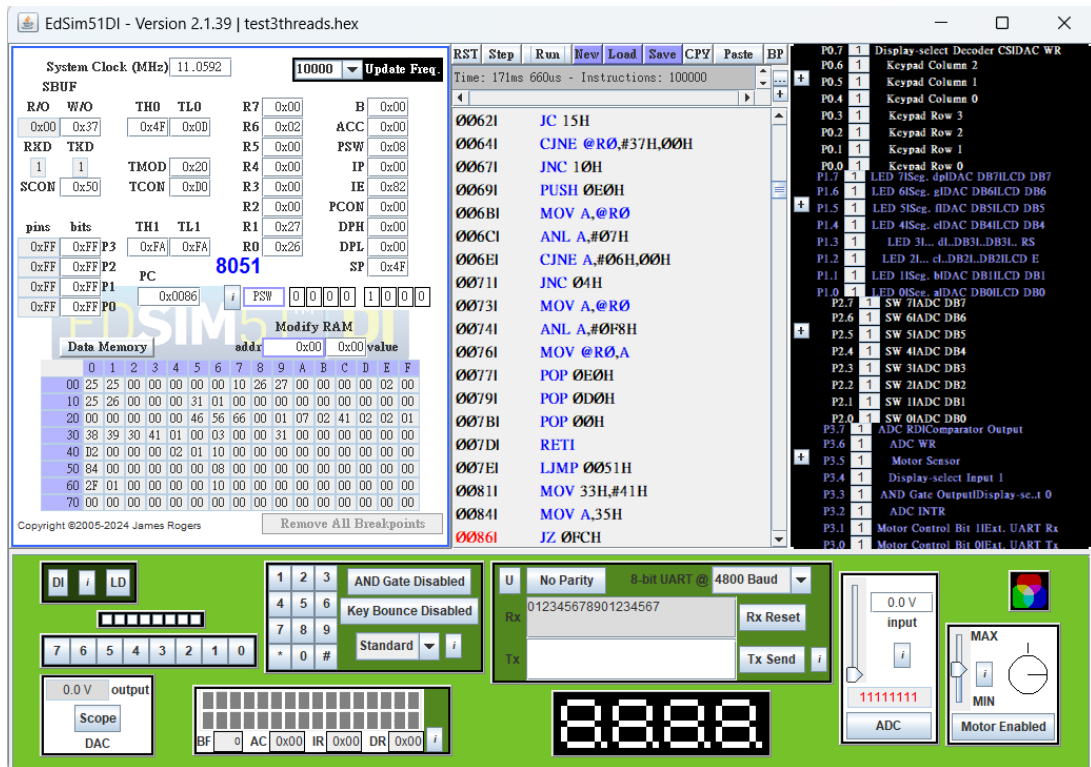


Figure 10: Unfair condition for [Producer1](#)

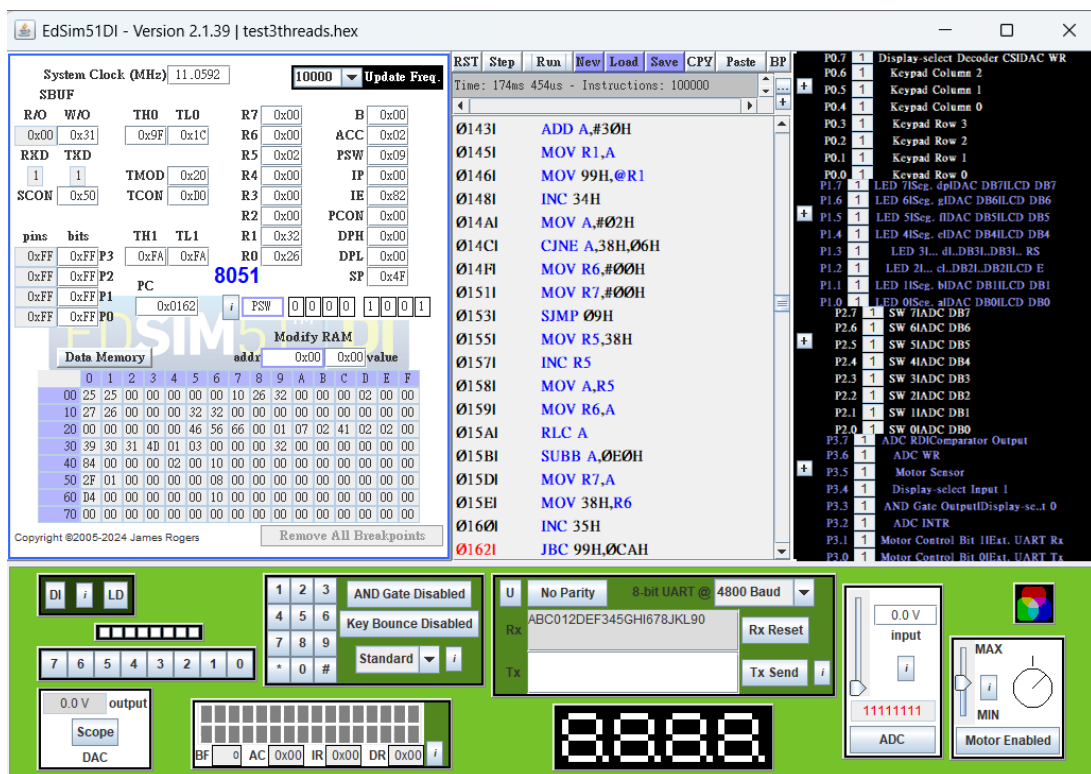


Figure 11: Unfairness resolved with the new scheduling

If round-robin scheduling is used, unfairness between the two producers will occur. When Thread 0 is assigned to [Producer1](#), Thread 1 to [Producer2](#), and Thread 2 to [Consumer](#),

the execution sequence is $Producer1 \rightarrow Producer2 \rightarrow Consumer \rightarrow Producer1$. However, since the timer interval is long enough for **Producer1** to fill the entire buffer within one time slice, **Producer2** gets blocked at the semaphore. After **Consumer** empties the buffer, **Producer1** fills it again, and **Producer2** never gets a chance to write. The UART output shows only uppercase letters.

Conversely, if **Thread 0** is assigned to **Producer2** and **Thread 1** to **Producer1**, then **Producer1** never gets a chance to write, and the UART output shows only digits.

To solve this, the round-robin scheduling policy was replaced with a ping-pong policy. The thread index first counts up to **Thread 3** (which does not exist), then reverses and counts down to **Thread 0**, producing the sequence:

$Thread\ 0 \rightarrow Thread\ 1 \rightarrow Thread\ 2 \rightarrow Thread\ 3 \rightarrow Thread\ 2 \rightarrow Thread\ 1 \rightarrow Thread\ 0 \rightarrow \dots$

Since **Thread 3** does not exist, the scheduler skips it, and the effective execution sequence becomes:

$Producer1 \rightarrow Consumer \rightarrow Producer2 \rightarrow Consumer \rightarrow Producer1 \rightarrow \dots$

This ensures **Consumer** is always scheduled between the two producers, giving both equal access to the buffer and eliminating starvation. The UART output shows letters and digits alternating (e.g., ABC012DEF345GHI678JKL90).